



SyncHR

Project Report

Submitted to: **Professor Letterio Galletta**

A Distributed Web-Based Human Resource Management System

Built with Go (backend) and React (frontend)

Final Project — Distributed Programming for Web, IoT, and Mobile Systems

University of Florence

Syed Mohammad Askari Abidi | 7178458

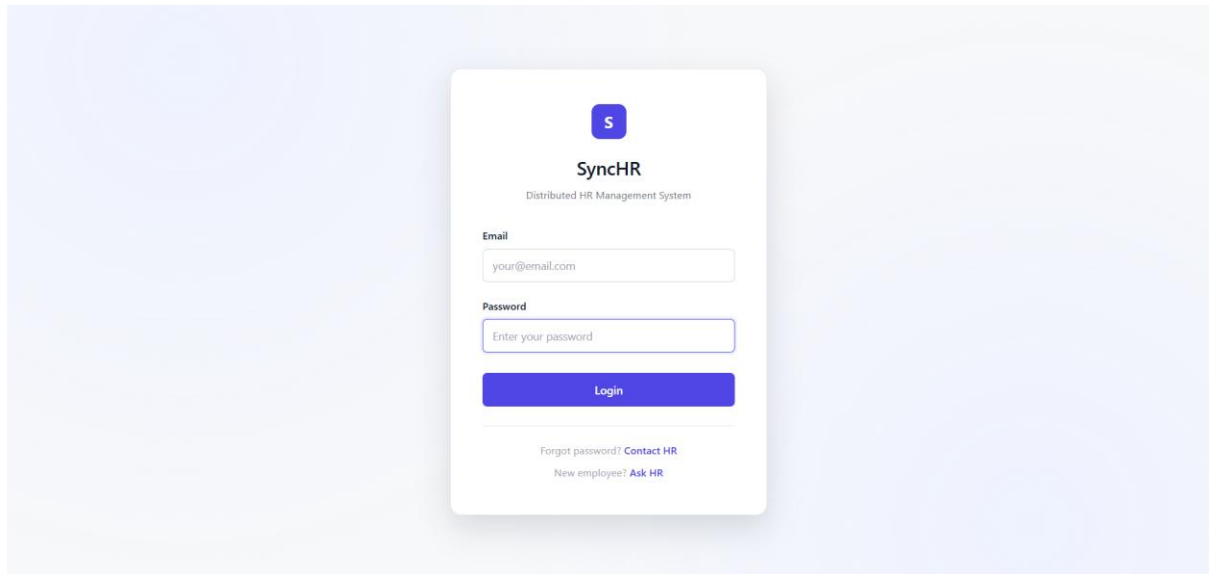
Syed.abidi@edu.unifi.it

TABLE OF CONTENTS

Introduction.....	2
Project Design & Architecture	2
Why Client-Server, and Why Two Separate Deployables	2
The Backend: One Process, Many Concurrent Goroutines	3
Authentication: Stateless by Design	3
The Database Layer	4
The Frontend: Context + Hooks, One WebSocket for the Whole App	4
Technologies & Tools Used.....	4
Distributed Programming Topics Covered	5
Client-Server Architecture with REST APIs	5
Real-Time Communication via WebSocket.....	5
JWT-Based Stateless Authentication with RBAC	6
Asynchronous, Concurrent Processing with Goroutines and Channels.....	6
Modular, Testable System Design	6
Use Case Walkthroughs	6
Use Case: Employee Applies for Leave, HR Approves, Employee Is Notified in Real Time	6
Actors	6
Preconditions	6
Main Flow	7
Postconditions	7
Alternate Flow — Rejection	7
Alternate Flow — Insufficient Balance	8
Use Case: HR-Mediated Password Reset	8
Main Flow	8
Development Process & Notable Engineering Decisions.....	9
Bugs Found Through Testing, Not Assumed Away	9
A Deliberate Scope Decision: HR-Mediated Password Reset.....	10
Verifying Deployment, Not Just Writing Configuration	10
Testing & Quality Assurance	10
Known Limitations & Future Work.....	11
Conclusion	11
References	11

INTRODUCTION

SyncHR is a distributed Human Resource Management System built to demonstrate real-time, concurrent, client-server design in a domain most people already understand intuitively: everyday HR operations. An employee checks in for the day, applies for leave, and looks at a payslip; an HR manager reviews that leave request and approves it. The interesting engineering problem is not the CRUD, it is what happens the instant that approval is clicked: the employee, if they happen to have the app open in another tab, sees it immediately, with no refresh and no polling.



That single requirement, instant, bidirectional, many-to-many communication between a Go backend and any number of React clients is what pulls in most of the distributed-systems material this project set out to demonstrate: concurrent goroutines, channel-based coordination instead of locks, a WebSocket protocol layered on top of a REST API, and the stateless JWT authentication that has to keep working across both. (Gorilla WebSocket - Real-time Communication, n.d.) (React - Official Documentation and Guide, n.d.)

PROJECT DESIGN & ARCHITECTURE

WHY CLIENT-SERVER, AND WHY TWO SEPARATE DEPLOYABLES

The backend and frontend are two independently buildable, independently deployable artifacts that only ever talk to each other over the network (HTTP and WebSocket) never through shared memory, a shared filesystem, or a shared process. This is a deliberate, if small-scale, illustration of the client-server model that underlies almost every distributed system: the two sides can be developed, tested, scaled, and deployed on entirely different machines without either one knowing anything about the other's internals, only its public contract (the REST API and the WebSocket message format). (RESTful API Design Best Practices and Guidelines, n.d.)

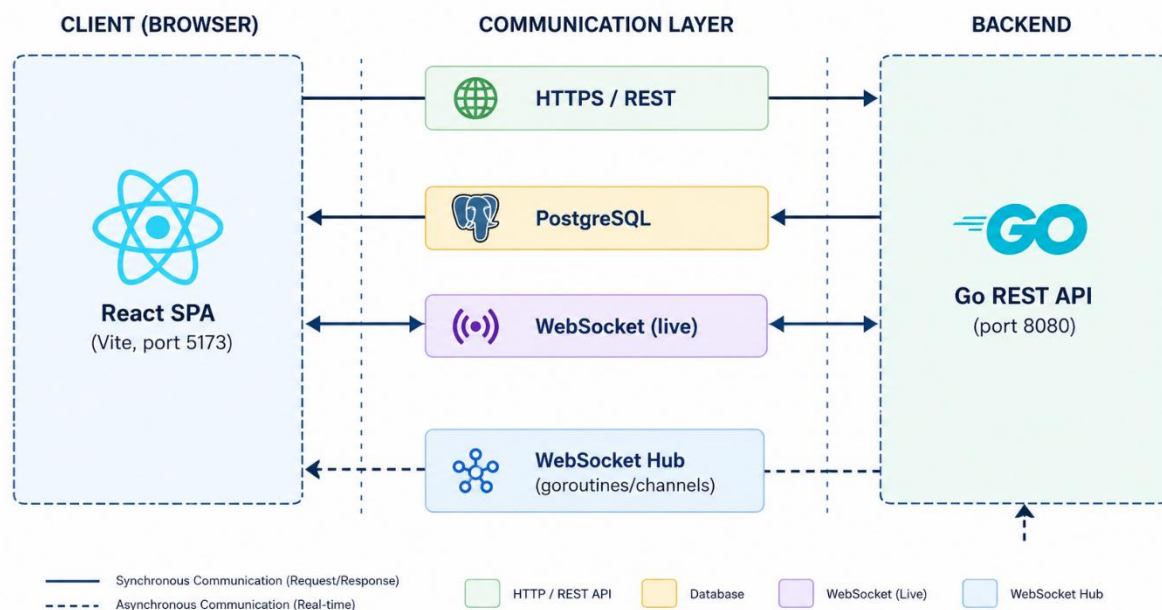
THE BACKEND: ONE PROCESS, MANY CONCURRENT GOROUTINES

Go's HTTP server already gives every incoming request its own goroutine for free; the design decision that mattered here was what to do with the WebSocket connections, which are long-lived and need to be pushed to from arbitrary other goroutines (any HTTP handler that wants to notify someone). (The Go Programming Language - Official Documentation, n.d.)

The chosen design is the classic Go "hub" or "actor" pattern:

- ✓ Every WebSocket connection gets its own read-loop goroutine (*readPump*), whose only job is to detect when the client disconnects.
- ✓ A single, dedicated goroutine (*Hub.run*) owns a map of connected clients. It is the only code in the entire program that ever reads or writes that map.
- ✓ Every other goroutine — the *readPumps*, and every HTTP handler that wants to deliver a notification — talks to the hub exclusively through three channels: register, unregister, and direct (send to one user or one role).

The payoff of this design is that there is no mutex anywhere in the notification system, and yet it is fully safe under concurrent access from dozens of goroutines at once, a property that was not just assumed but actually verified: a test spins up 20 real WebSocket connections and messages all of them concurrently under Go's race detector (go test -race), and the suite is clean.



AUTHENTICATION: STATELESS BY DESIGN

Every protected request carries a JSON Web Token signed with HMAC-SHA256. The server never stores session state — anyone with a valid, unexpired, correctly-signed token is authenticated, and their role (*employee* or *hr_manager*) travels inside the token itself. This matters for distribution specifically: a stateless token means any number of backend instances could validate the same request without needing to share a session store, which is the standard approach for authentication that must scale horizontally.

The one place this app deliberately breaks the "just check the Authorization header" pattern is the WebSocket handshake: browsers cannot attach custom headers to a WebSocket upgrade request, so the token travels as a query parameter instead and is verified with the exact same signature-checking code path as the HTTP middleware, just invoked differently.

THE DATABASE LAYER

PostgreSQL, accessed directly through Go's database/sql with the lib/pq driver — no ORM. Every query is hand-written SQL, which keeps the exact shape of every read and write fully visible and made it straightforward to track down (and fix) a real timezone bug where a Go time.Time in the server's local timezone was written into a TIMESTAMP WITHOUT TIME ZONE column and read back misinterpreted as UTC, silently skewing attendance durations. Foreign keys use ON DELETE CASCADE so that deleting a user cleanly removes their attendance, leave, payslip, and notification rows without the application having to orchestrate that itself.

THE FRONTEND: CONTEXT + HOOKS, ONE WEBSOCKET FOR THE WHOLE APP

The React app opens exactly one WebSocket connection, owned by a *NotificationProvider* mounted once near the root of the component tree (inside *AuthProvider*, outside the router), so it persists across every route change and is torn down and reopened automatically whenever the logged-in account changes. Auth state and notification state are each exposed through a small custom hook (*useAuth*, *useNotifications*) rather than being read out of context objects directly in every component, which keeps the 8+ pages that need "who is logged in" from needing to know anything about how that state is actually stored.

A second, smaller piece of shared frontend logic — *useDateNavigator* — encapsulates the month-navigation/day-tabs/date-picker behavior used identically by both the employee's own attendance view and HR's all-employee attendance view, so that behavior (and its bugs, and its fixes) exist in exactly one place. (Gorilla Web Toolkit - Mux Package, n.d.) (Axios - Promise Based HTTP Client, n.d.)

TECHNOLOGIES & TOOLS USED

Layer	Technology	Role in this project
Backend language	Go	REST API, WebSocket server, all business logic
HTTP routing	gorilla/mux	Route table, path parameters, middleware chaining
Real-time	gorilla/websocket	WebSocket handshake and framing under the hub
Auth	golang-jwt/jwt v5	Signing and verifying JWTs
Password security	golang.org/x/crypto/bcrypt	Password hashing (and temporary-password hashing on reset)

Layer	Technology	Role in this project
Database driver	lib/pq	PostgreSQL connectivity via database/sql
Database	PostgreSQL 16	Persistent storage, 9 tables, cascading foreign keys
Frontend framework	React 19	Component-based SPA UI
Routing	React Router 7	Client-side routing, role-aware route guards
Build tool	Vite	Dev server, production bundling
HTTP client	Axios	REST calls with interceptors for auth headers and 401 handling
Real-time client	Native browser WebSocket API	No client library needed
Containerization	Docker + Docker Compose	Three-service orchestrated deployment
Static file serving	nginx	Serves the built frontend in production, with SPA fallback routing
Testing	Go's built-in testing package	Unit, integration, and concurrency tests — no third-party test framework

DISTRIBUTED PROGRAMMING TOPICS COVERED

CLIENT-SERVER ARCHITECTURE WITH REST APIs

The entire backend surface is a REST API (see the companion Documentation for the full endpoint reference): resource-oriented URLs, HTTP verbs mapped to CRUD-style operations, JSON request/response bodies, and standard status codes (200/201/400/401/403/404/500) used consistently to communicate outcome to the client.

REAL-TIME COMMUNICATION VIA WEBSOCKET

Beyond the request/response REST model, the app maintains long-lived, full-duplex WebSocket connections for server-initiated push. Three distinct real-time flows are implemented and were each individually verified against a live server: an employee applying for leave pushes instantly to every connected HR manager; HR approving or rejecting a request, pushes instantly to that specific employee; and HR broadcasting an announcement pushes instantly to every connected employee.

JWT-BASED STATELESS AUTHENTICATION WITH RBAC

Covered in depth in section 2.3. Role-based access control is enforced twice, deliberately redundantly: once on the backend (every HR-only handler checks the role claim and returns 403 otherwise — this is the real security boundary), and again on the frontend (route guards redirect a mismatched role away from a page before it even renders, purely for user experience, since a determined client could always call the API directly). (golang-jwt - JSON Web Tokens for Go, n.d.)

ASYNCHRONOUS, CONCURRENT PROCESSING WITH GOROUTINES AND CHANNELS

Covered in depth in section 2.2. This is the part of the project most directly aimed at the course's core material: the hub is not a simulation or a toy example of Go concurrency, it is the actual mechanism carrying real notification traffic in the running application, and its correctness under concurrent load is asserted by an automated test, not just believed.

MODULAR, TESTABLE SYSTEM DESIGN

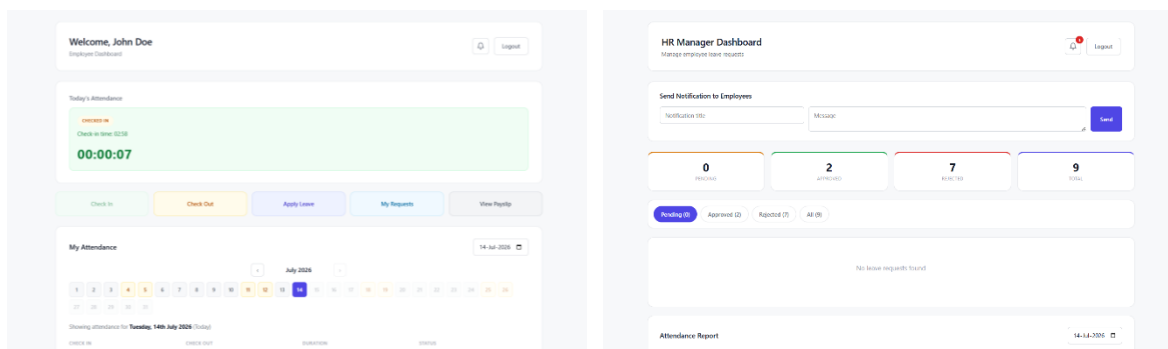
The backend's router construction was deliberately factored out of *main()* into a standalone *SetupRouter(db)* function precisely so that the test suite could build the exact same route table, wired to a real database, without ever needing to start a real network listener — the same design discipline that makes a system easier to distribute across multiple deployment targets also made it possible to write true integration tests that exercise the real HTTP handler chain.

USE CASE WALKTHROUGHS

USE CASE: EMPLOYEE APPLIES FOR LEAVE, HR APPROVES, EMPLOYEE IS NOTIFIED IN REAL TIME

Actors

- ✓ Employee — a logged-in user with role "*employee*".
- ✓ HR Manager — a logged-in user with role "*hr_manager*", with the HR Dashboard open in a browser tab.



Preconditions

- ✓ Both the employee and the HR manager are authenticated (each holds a valid JWT).
- ✓ The employee has a positive leave balance for at least one leave type.
- ✓ The HR manager's browser has an open WebSocket connection (established automatically on login by *NotificationProvider*).

Main Flow

1. The employee opens Apply Leave, selects a leave type (fetched live from GET /leave/types), picks a start and end date, and submits.
2. The frontend sends POST /leave/apply with the leave type, dates, computed number of days, and an optional reason.
3. The backend validates the request, checks the employee's current balance for that leave type, and — if sufficient — inserts a new leave_request row with status "pending" and returns 201 Created with the new request's ID.
4. Immediately after the insert succeeds, the handler calls `notifyRole("hr_manager", ...)`, which looks up every user with the HR manager role, persists one notification row per HR manager, and pushes each one live over the WebSocket hub to whichever of those HR managers happen to be currently connected.
5. The HR manager's browser receives the WebSocket message, the *NotificationContext* prepends it to the in-memory notification list, and the bell icon's unread badge updates — with no page refresh and no polling interval involved.
6. The HR manager opens the notification bell (or simply navigates to the Leave Requests section, which is refetched on demand) and reviews the new request: employee name, leave type, dates, number of days, and reason.
7. The HR manager clicks Approve, enters a required approval note, and confirms. The frontend sends `PUT /leave/approve/{id}`.
8. The backend re-verifies the requester is an HR manager, updates the request's status to "approved", deducts the approved number of days from the employee's leave_balance row, and calls `notifyUser(employeeID, ...)` to persist and push a "Leave Approved" notification to that specific employee.
9. If the employee's browser is open, their notification bell updates instantly, showing the approval. If not, the notification is waiting for them (persisted in the database) the next time they open the app and fetch GET /notifications.

Postconditions

- ✓ The leave_request row's status is "approved", with *approved_by*, *approval_date*, and *approval_notes* populated.
- ✓ The employee's leave_balance for that leave type is reduced by the approved number of days.
- ✓ Two notification rows exist in the database (one to HR at submission time, one to the employee at approval time), independent of whether either party was online to receive the live push.

Alternate Flow — Rejection

Identical up through step 6. Instead of approving, HR clicks Reject with a required reason. `PUT /leave/reject/{id}` sets status to "rejected" (no balance is deducted) and notifies the employee with a "Leave Rejected" notification, delivered the same way.

My Leave Requests

Track all your leave applications and their status

Back to Dashboard

All Requests

Pending

Approved

Rejected

LEAVE TYPE	START DATE	END DATE	DAYS	STATUS	REASON	HR COMMENTS	SUBMITTED
Casual Leave	2026-07-31T00:00:00Z	2026-08-08T00:00:00Z	9	REJECTED	gg	no reason	7/12/2026
Casual Leave	2026-07-24T00:00:00Z	2026-07-25T00:00:00Z	2	REJECTED	no request	no	7/12/2026
Casual Leave	2026-07-14T00:00:00Z	2026-07-15T00:00:00Z	2	REJECTED	no reason	no reason	7/12/2026
Casual Leave	2026-07-13T00:00:00Z	2026-07-14T00:00:00Z	2	REJECTED	personal work	no reason	7/12/2026
Casual Leave	2026-07-13T00:00:00Z	2026-07-14T00:00:00Z	2	REJECTED	personal work	no reason	7/12/2026
Casual Leave	2026-07-13T00:00:00Z	2026-07-14T00:00:00Z	2	REJECTED	personal work	no reason	7/12/2026

0
PENDING

2
APPROVED

6
REJECTED

8
TOTAL

Alternate Flow — Insufficient Balance

If the employee requests more days than their remaining balance for that leave type, step 3 rejects the request at *POST /leave/apply* time with a 400 Bad Request before any row is ever written, the employee sees an inline error and no HR manager is notified of a request that was never actually created.

Insufficient leave balance for this request

Leave Type *

Casual Leave

Start Date *

16-Jul-2026

End Date *

12-Nov-2026

Number of Days

120

Automatically calculated

USE CASE: HR-MEDIATED PASSWORD RESET

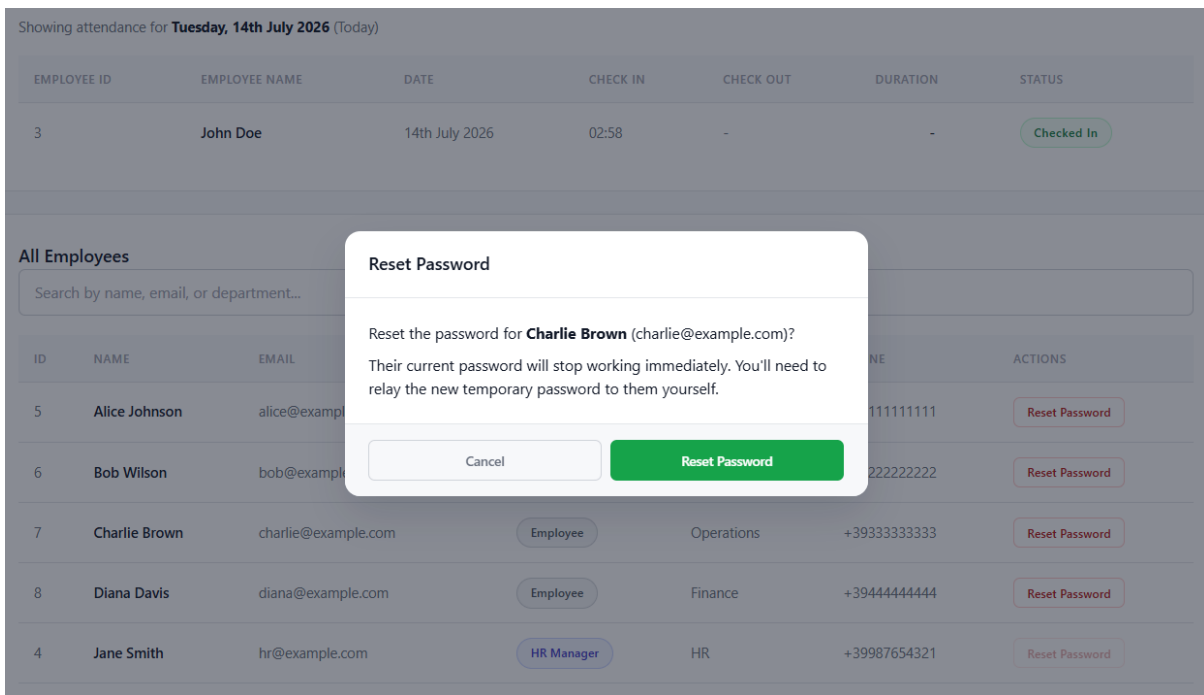
A secondary use case chosen specifically because there is no self-service "forgot password" flow in this system by design: resets are always mediated by HR, which was a deliberate scope decision (see section 6) to avoid depending on a real email/SMTP service that would not reliably work in a grading environment.

Main Flow

1. An employee who forgot their password clicks "Forgot password? Contact HR" on the login page, which opens a pre-filled email to HR through the browser's default mail client.
2. HR opens the Employee Directory on the HR Dashboard, finds the employee, and clicks "Reset Password".
3. A confirmation dialog warns that the employee's current password will stop working immediately. HR confirms.
4. The frontend sends *PUT /users/{id}/reset – password*. The backend verifies the caller is HR, generates a new 12-character password using Go's crypto/rand (a

cryptographically secure source, not math/rand, since this value is handed to a human to relay), hashes it with bcrypt, and overwrites the employee's password_hash.

5. The backend also fires an in-app notification to the employee ("HR has reset your password. Please contact HR to receive your new temporary password."), informative, but deliberately without the password itself, which never travels through the notification system.
6. The new plaintext temporary password is returned once, directly to HR, in a modal with a copy-to-clipboard button, and is never persisted or retrievable again after that response.
7. HR relays the new password to the employee out of band (chat, phone, in person), who logs in and is expected to be able to change it via their profile.



Showing attendance for **Tuesday, 14th July 2026** (Today)

EMPLOYEE ID	EMPLOYEE NAME	DATE	CHECK IN	CHECK OUT	DURATION	STATUS
3	John Doe	14th July 2026	02:58	-	-	Checked In

All Employees

Search by name, email, or department...

ID	NAME	EMAIL	ROLE	DEPARTMENT	PHONE	ACTIONS
5	Alice Johnson	alice@example.com	Employee	Operations	+1111111111	Reset Password
6	Bob Wilson	bob@example.com	Employee	Operations	+2222222222	Reset Password
7	Charlie Brown	charlie@example.com	Employee	Operations	+3933333333	Reset Password
8	Diana Davis	diana@example.com	Employee	Finance	+3944444444	Reset Password
4	Jane Smith	hr@example.com	HR Manager	HR	+39987654321	Reset Password

DEVELOPMENT PROCESS & NOTABLE ENGINEERING DECISIONS

BUGS FOUND THROUGH TESTING, NOT ASSUMED AWAY

Writing a real automated test suite against a real database, rather than trusting the code by inspection, surfaced three genuine, previously-invisible bugs, each of which would have been embarrassing to discover live in front of a grader:

- ✓ Login compared a bcrypt hash directly against the plaintext submitted password instead of calling `bcrypt.CompareHashAndPassword`, meaning any account created through the real registration endpoint could never subsequently log in. The only reason two demo accounts worked throughout earlier manual testing was that their `password_hash` column had been seeded with the literal plaintext string "password123", not a real hash.
- ✓ A timezone mismatch between Go's `time.Now()` (server-local timezone) and Postgres's timezone-less `TIMESTAMP` column produced attendance durations that could come out negative for a check-in immediately followed by a check-out.

- ✓ The frontend hardcoded leave type names against IDs 1 through 3 in two separate places, even though the backend seeds five leave types, Maternity and Paternity leave were both silently unreachable in the UI.

All three are fixed and each now has a dedicated regression test that fails if the bug is ever reintroduced.

A DELIBERATE SCOPE DECISION: HR-MEDIATED PASSWORD RESET

Four options were considered for "forgot password": a full token-based email-reset flow with real SMTP delivery, a security-question-based self-service reset, a simulated-email flow (real reset tokens but displaying the link in the UI instead of emailing it), and HR-mediated reset. HR-mediated reset was chosen deliberately, it requires zero external service dependency, so the feature works identically in every environment (a laptop, a Docker container, a grading machine) without needing real SMTP credentials that might not be configured, and it mirrors how many small organizations actually handle this in practice.

VERIFYING DEPLOYMENT, NOT JUST WRITING CONFIGURATION

The Docker Compose setup was not written and left untested. Docker Desktop was used and full three-container stack was built from a completely empty Postgres volume, and a real register → login flow was exercised directly against the containerized backend to confirm the schema auto-applied correctly and the bcrypt fix held up in a clean environment with no manually-seeded data. (PostgreSQL Official Documentation, n.d.)

TESTING & QUALITY ASSURANCE

27 automated Go tests, organized into three layers:

- ✓ Unit tests for JWT validation (*middleware_test.go*) valid tokens, wrong signing secret, expired tokens, malformed tokens, and the classic "*alg: none*" signature-bypass attempt.
- ✓ WebSocket hub tests (*hub_test.go*) that dial real client connections against a real httptest server and verify targeted delivery, role-based delivery, resilience to a peer disconnecting, and 20 concurrent clients with no data race.
- ✓ HTTP integration tests (*integration_test.go*) that exercise the real router with a real database: registration/login, RBAC on every role-gated endpoint, the full attendance check-in/check-out cycle, and the full leave apply → approve/reject → notification flow.

The full suite passes under GO test -race with zero data races reported, a meaningful claim specifically because the system's core real-time mechanism is concurrent by construction, not incidentally.

On the frontend, npm run lint runs cleanly with zero errors (down from 25 real errors caught during a full-repo audit, dead React imports left over from an older JSX transform, hook-ordering issues, and two context files that mixed component and non-component exports in a way that hurts React's Fast Refresh).

```
VS 1.4 ready in 247 ms
C:\Windows\System32\cmd.exe
ole: hr_manager
2026/07/14 03:11:11 🍌 JWT Middleware - Extracted UserID: 244, Email: test_employee_1783991468750540200@example.com, Rol
e: employee
2026/07/14 03:11:11 🍌 JWT Middleware - Extracted UserID: 244, Email: test_employee_1783991468750540200@example.com, Rol
e: employee
--- PASS: TestLeave_FullApprovalFlow (2.66s)
=== RUN TestLeave_RejectFlow
2026/07/14 03:11:14 🍌 JWT Middleware - Extracted UserID: 246, Email: test_employee_1783991471409176800@example.com, Rol
e: employee
2026/07/14 03:11:14 🍌 JWT Middleware - Extracted UserID: 246, Email: test_employee_1783991471409176800@example.com, Rol
e: employee
2026/07/14 03:11:14 🍌 JWT Middleware - Extracted UserID: 247, Email: test_hr_manager_1783991472728733200@example.com, R
ole: hr_manager
2026/07/14 03:11:14 🍌 JWT Middleware - Extracted UserID: 246, Email: test_employee_1783991471409176800@example.com, Rol
e: employee
--- PASS: TestLeave_RejectFlow (2.65s)
=== RUN TestParseJWT_ValidToken
--- PASS: TestParseJWT_ValidToken (0.00s)
=== RUN TestParseJWT_WrongSecret
--- PASS: TestParseJWT_WrongSecret (0.00s)
=== RUN TestParseJWT_ExpiredToken
--- PASS: TestParseJWT_ExpiredToken (0.00s)
=== RUN TestParseJWT_MalformedToken
--- PASS: TestParseJWT_MalformedToken (0.00s)
=== RUN TestParseJWT_RejectsNoneAlgorithm
--- PASS: TestParseJWT_RejectsNoneAlgorithm (0.00s)
PASS
ok      github.com/askariabidi/SynchR  31.399s
D:\DP\SynchR\backend>
```

KNOWN LIMITATIONS & FUTURE WORK

- ✓ Single-instance WebSocket hub — the connected-client registry lives in one process's memory. Running multiple backend instances behind a load balancer would require a shared pub/sub layer (e.g., Redis) so a notification triggered on one instance reaches a client connected to another.
- ✓ No self-service password reset — a deliberate trade-off (section 6.2), but a real SMTP-backed reset-token flow would be the natural next step for production deployment.
- ✓ CORS and the WebSocket origin check are currently permissive (development defaults) and should be tightened before any real public deployment. (MDN Web Docs - Cross-Origin Resource Sharing (CORS), n.d.)
- ✓ No automated end-to-end browser tests (e.g., Playwright) the manual/UI test cases in the companion documentation are currently exercised by hand; automating them would close the last gap in the test pyramid.

CONCLUSION

SynchR started as an HR management CRUD application and, over the course of development, grew a genuine distributed-systems core: a hand-built, concurrency-safe real-time notification layer backing three real product flows, stateless JWT authentication enforced on both sides of the client-server boundary, and a fully containerized, independently verified deployment story. Just as importantly, the project was treated as something to be tested and verified rather than assumed correctly, which is precisely how three real, non-obvious bugs were found and fixed before they could surface during a live demonstration.

REFERENCES

Axios - Promise Based HTTP Client. (n.d.). Retrieved from <https://axios-http.com/>

golang-jwt - JSON Web Tokens for Go. (n.d.). Retrieved from <https://github.com/golang-jwt/jwt>

Gorilla Web Toolkit - Mux Package. (n.d.). Retrieved from <https://github.com/gorilla/mux>

Gorilla WebSocket - Real-time Communication. (n.d.). Retrieved from <https://github.com/gorilla/websocket>

MDN Web Docs - Cross-Origin Resource Sharing (CORS). (n.d.). Retrieved from <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>

PostgreSQL Official Documentation. (n.d.). Retrieved from <https://www.postgresql.org/docs/>

React - Official Documentation and Guide. (n.d.). Retrieved from <https://react.dev/>

RESTful API Design Best Practices and Guidelines. (n.d.). Retrieved from <https://restfulapi.net/>

The Go Programming Language - Official Documentation. (n.d.). Retrieved from <https://golang.org/doc/>